Assignment 3:

Q1.Maximum Likelihood Estimation

Instructions: This assignment engages you in the practical application of maximum likelihood estimation for linear regression modeling. Follow the steps carefully, and submit your work as a single PDF file that includes all required code and graphs. **Attach** supplementary code files (.py and .ipynb) as used in this assignment.

Objective: Learn how to fit a linear regression model using maximum likelihood estimation and understand the effects of model adjustments, such as adding a bias term.

Step 1: Graphing the Relationship

```
import numpy as np
import scipy.linalg
import matplotlib.pyplot as plt

X = np.array([-3, -1, 0.0, 1, 3]).reshape(-1,1) # 5x1 vector, N=5, D=1
y = np.array([2.2, 3.7, 3.14, 3.67, 4.67]).reshape(-1,1) # 5x1 vector
```

- Using the provided code, plot the relationship between variables `x` and `y`.
- Attach the Python code used to generate the graph in your PDF.
- Display the graph you obtain.

Step 2: Applying the Derived Equation

$$\boldsymbol{\theta}_{\mathrm{ML}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

- Implement the final equation we derived in class to compute the parameter Theta (θ).
    - Refer to the Python file attached with this assignment for the implementation guide.
- Display the resulting Theta value and attach the code used to obtain it.
- Attach the Python code used in your PDF.

Step 3: Plotting the Initial Model

- Apply the Theta obtained from Step 2 to a linear equation.
- Plot this equation on the same graph as in Step 1.
- Ensure the graph displays the original 5 data points and the linear model representing the obtained Theta.
- Attach the Python code used to generate the graph in your PDF.

Step 4: Improving the Model

- You will notice that the linear model does not fit well with the training points provided. This is because we did not include the bias of the linear equation when calculating Theta initially.
- Enhance your model by including a bias term. Stack a column of ones vertically with your x data (as shown in the provided code snippet).

```
# ================================================================================
print("="*10)
N, D = X.shape
X_aug = np.hstack([np.ones((N,1)), X]) # augmented training inputs of size N x (D+1)
print("X_aug = ")
print(X_aug)
print("="*10)
# ================================================================================
```

-

```
==========
X_aug =
[[ 1. -3.]
 [ 1. -1.]
 [ 1.  0.]
 [ 1.  1.]
 [ 1.  3.]]
==========
```

- Recalculate Theta using this new matrix configuration.
- The resulting Theta should now have two values (shape (2,1)).
- Attach your code and display both values of Theta.
- Explain how these values differ from those obtained in Step 2.

Step 5: Plotting the Improved Model

- Use the new Theta values from Step 4 in your linear equation.
- Plot this updated model on the same graph as in Step 1.
- The graph should still display the original 5 data points, along with the new linear model.
- Attach the Python code used to generate the graph in your PDF.

Step 6: Analytical Explanation

- Explain why the addition of a column of ones in Step 4 (bias term) and subsequent recalculation improves the fit of the model to the data.

Q2: Maximum Likelihood Estimation with Features

Instructions: This assignment engages you in the practical application of maximum likelihood estimation with features for linear regression modeling. Follow the steps carefully, and submit your work as a single PDF file that includes all required code and graphs. **Attach** supplementary code files (.py and .ipynb) as used in this assignment.

Objective: Learn how to fit a linear regression model using maximum likelihood estimation with features polynomial and understand the effects of model adjustments, such as adding a degree of polynomial.

Step-by-Step Instructions

Q1. Maximum Likelihood Estimation with Features

Step 1: Understanding and Plotting the Data

```
import numpy as np
import matplotlib.pyplot as plt

# Data Initialization
X = np.array([-4.5, -3.5, -3, -1.8, -0.2, 0.3, 1.3, 2.6, 3.8, 4.8]).reshape(-1,1)
y = np.array([
    [-0.91650116],
    [-0.47546053],
    [-0.10972425],
    [0.29504095],
    [-0.01596218],
    [0.10014949],
    [0.48104303],
    [0.10979023],
    [-0.99742128],
    [-0.91221826]
]).reshape(-1,1)

X_test = np.array([-3.99, -1.38, -1.37, -0.94,
  0.69,  1.4,   1.57,  1.78,  1.81,  4.89]).reshape(-1,1)
y_test = np.array([
    [-0.80737607],
```

```
    [0.19813376],
    [0.19537639],
    [0.07185977],
    [0.24954213],
    [0.50662504],
    [0.52943298],
    [0.52406997],
    [0.51999057],
    [-0.82318288]
]).reshape(-1,1)
```

- Using the provided code, plot the relationship between variables `x` and `y`.
- Attach the Python code used to generate the graph in your PDF.
- Display the graph you obtain.

Step 2: Polynomial Feature Transformation

- Implement the poly_features function to transform input data X into polynomial features of degree K.

(Polynomial Regression)

$$\phi(x) = \begin{bmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \\ \phi_{K-1}(x) \end{bmatrix} = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \\ \vdots \\ x^{K-1} \end{bmatrix}$$

(Feature Matrix for Second-order Polynomials)

$$\mathbf{\Phi} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_N & x_N^2 \end{bmatrix}$$

- Attach the Python code used for the implementation.
- Explain the purpose and the effect of this transformation.

Step 3: Fitting the Model Using Maximum Likelihood

- Using the poly_features function from Step 2, transform both training and test datasets for a degree of polynomial 4.
- Apply the nonlinear_features_maximum_likelihood function to estimate model parameters (Theta) with Phi from training data.

$$\theta_{\mathrm{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top y$$

- Attach the Python code used for the implementation and show their result.

Step 4: Model Evaluation Using RMSE

- Implement the RMSE function to evaluate the accuracy of your model on both the training and test datasets.
- Calculate RMSE for models with polynomial degrees ranging from 0 to 15.
- Plot these RMSE values against the polynomial degree for both training and test datasets.
- Attach the Python code used for the implementation and generate the graph in your PDF.

Step 5: Selecting the Best Model

- Identify the polynomial degree that results in the lowest RMSE for the test dataset.
- Using the optimal degree, plot the model's predictions against the original data points.

```
X_test_all = np.linspace(-5, 5, 100).reshape(-1,1)
Phi_test_all = poly_features(X_test_all, optimal_k)
ypred_test_all = Phi_test_all @ theta_optimal
```

- Discuss why this particular model degree was optimal and how model complexity affects overfitting and underfitting.
- Attach the Python code used for the implementation and generate the graph in your PDF.

Step 6: Conclusion

- Summarize your findings regarding the polynomial regression model's performance.
- Discuss the trade-offs between model complexity (degree of the polynomial) and prediction accuracy.

Submission Requirements:

- Submit your completed assignment as a PDF containing all necessary code snippets and graphs.
- Attach corresponding .py and .ipynb files containing executable code.
- Ensure all files are clearly labeled and organized.
- Late submissions will be subject to deductions according to the policy.

Evaluation Criteria: Your assignment will be graded based on the accuracy of your implementation, the clarity of your presentation, and the completeness of your submission. Specific attention will be given to how well you follow these guidelines:

- **Step-by-Step Explanation:** This assignment requires clear, step-by-step explanations. You must explicitly label and explain each step (e.g., Step 1, Step 2, Step 3, Step 4). Failure to provide detailed explanations for each required step will result in a proportional deduction in your score. For example, if the assignment requires four steps and you complete only three, this will result in a score of 75 out of 100.